Line-by-Line Explanation of VFL_SHAP.ipynb

This notebook implements **Vertical Federated Learning (VFL) with SHAP explanations** for DDoS attack detection using network flow data. The code demonstrates how to explain which party (data owner) contributes most to attack detection decisions.

Overview: Why These Techniques?

Why Vertical Federated Learning (VFL)?

- **Privacy Preservation**: In real-world scenarios, different organizations (RAN operators, cloud providers, enterprises) cannot share raw network data due to privacy regulations (GDPR, HIPAA) and competitive concerns
- **Data Distribution**: Each party has different features about the same entities (network flows), but cannot combine them directly
- **Collaborative Learning**: VFL allows parties to train a model together without exposing raw data - only embeddings are shared
- **Use Case**: Network security requires insights from multiple network segments, but data cannot be centralized

Why SHAP Explanations?

- **Interpretability**: Deep learning models are "black boxes" - SHAP explains which features/parties drive predictions
- **Accountability**: In federated settings, parties need to understand their contribution to decisions
- **Actionable Insights**: SHAP values identify which party should take mitigation actions
- **Trust**: Stakeholders need explanations to trust AI decisions in critical security applications

Why Party-Level Meta-Features?

- **Dimensionality Reduction**: VFL embeddings are high-dimensional (32D) - aggregating to party-level summaries makes SHAP computation feasible
- **Interpretability**: Party-level explanations are more actionable than individual feature explanations
- **Efficiency**: Computing SHAP on 3 party features is much faster than on hundreds of original features

Why Knowledge Distillation (Meta-Model)?

- **SHAP Compatibility**: SHAP works better with simpler models - the meta-model approximates the complex VFL model
- **Computational Efficiency**: KernelExplainer on a simple 3-input model is faster than explaining the full VFL architecture
- **Fidelity**: The meta-model learns to mimic VFL predictions, preserving decision patterns while being explainable

---

CELL 0: Main Implementation

Section 1: Imports and Setup

```
# vfl_shap_party_level_ieee_kernel_agentic.py
```

- **Line 1**: Comment indicating the original script name/purpose

```
import pandas as pd
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
```

- **Lines 2-6**: Import essential libraries:
  - `pandas`: Data manipulation (CSV reading, DataFrames)
  - `numpy`: Numerical operations
  - `torch`: PyTorch for deep learning
  - `nn`: Neural network modules
  - `optim`: Optimizers (Adam)

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, recall_score, f1_score
```

- **Lines 8-10**: Import sklearn utilities:
  - `StandardScaler`: Normalizes features (mean=0, std=1)
  - `train_test_split`: Splits data into train/test sets
  - Metrics: Accuracy, Recall, F1-score for evaluation

```
import shap  # pip install shap
```

- **Line 12**: Import SHAP library for model interpretability/explainability

---

Section 2: Load Dataset

```
# ----------------------------
# 1. Load CSV Dataset
# ----------------------------
df = pd.read_csv("CICIDS2017_train.csv")  # update path if needed
```

- **Line 18**: Loads the CICIDS2017 dataset (network intrusion detection dataset) into a pandas DataFrame

```
# Drop non-feature columns if needed (e.g., index, IDs)
df = df.drop(columns=["Flow ID", "Src IP", "Dst IP", "Timestamp"],
errors="ignore")
```

- **Line 21**: Removes identifier columns that aren't useful features:
  - `Flow ID`: Unique flow identifier
  - `Src IP`, `Dst IP`: IP addresses (identifiers, not features)
  - `Timestamp`: Time information
  - `errors="ignore"`: Don't error if columns don't exist

Why remove these columns?

- **Identifiers vs Features**: Flow ID, IPs, and timestamps are identifiers, not predictive features. They don't help classify attacks
- **Privacy**: IP addresses are sensitive PII - removing them aligns with privacy-preserving ML
- **Overfitting Risk**: Unique identifiers (Flow ID) would cause perfect memorization but zero generalization
- **Model Focus**: We want the model to learn patterns from flow characteristics, not memorize specific flows

---

Section 3: Define Vertical Feature Split

```
# -----------------------------
# 2. Define Vertical Feature Split
# -----------------------------
party1_features = [
    "Flow Duration", "Total Fwd Packets", "Total Backward Packets", "Total Length
of Fwd Packets"
]
```

- **Lines 26-29**: Defines features for **Party 1** (first data owner):
  - Flow duration and forward packet counts/lengths
  - Represents: RAN/ISP edge data

```
party2_features = [
    "Total Length of Bwd Packets", "Fwd Packet Length Max", "Fwd Packet Length
Min", "Fwd Packet Length Mean"
]
```

- **Lines 30-33**: Defines features for **Party 2** (second data owner):
  - Backward packet lengths and forward packet statistics
  - Represents: Cloud/datacenter data

```
party3_features = [
    "Bwd Packet Length Max", "Bwd Packet Length Min", "Bwd Packet Length Mean",
"Bwd Packet Length Std"
]
```

- **Lines 34-37**: Defines features for **Party 3** (third data owner):
  - Backward packet statistics (mean, min, max, std)
  - Represents: Enterprise/edge gateway data

```
label_col = "label"   # binary 0/1
```

- **Line 39**: Defines the target column name (binary classification: 0=benign, 1=attack)

```
party_names = ["Party 1", "Party 2", "Party 3"]
```

- **Line 41**: List of party names for labeling

```
party_feature_groups = [
    "Flow duration & Fwd pkts",
    "Fwd length statistics",
    "Bwd length statistics",
]
```

- **Lines 42-46**: Descriptive names for each party's feature group (for reporting)

```
# Agentic roles (for magazine narrative)
party_domains = [
    "RAN / ISP edge",              # Party 1
    "Cloud / datacenter",          # Party 2
    "Enterprise / edge gateway",   # Party 3
]
```

- **Lines 48-53**: Describes the "domain" or role of each party in the network infrastructure

```
party_actions = [
    "adjust flow timeouts / RAN rate limits",
    "reconfigure load balancer / autoscale VNFs",
    "tighten firewall / ACL on return-path traffic",
]
```

- **Lines 55-59**: Suggested mitigation actions for each party (used later for recommendations)

Section 4: Preprocess Features

```
# ----------------------------
# 3. Preprocess Features
# ----------------------------
scaler1 = StandardScaler()
scaler2 = StandardScaler()
scaler3 = StandardScaler()
```

- **Lines 64-66**: Creates separate scalers for each party (important: each party normalizes independently in VFL)

Why separate scalers for each party?

- **VFL Privacy**: In real VFL, parties cannot share statistics (mean, std) - each must normalize independently
- **Feature Scale Differences**: Different parties have different feature ranges (e.g., packet counts vs. durations)
- **Neural Network Stability**: Normalization prevents features with large scales from dominating gradients
- **Convergence**: Scaled features help neural networks converge faster and more reliably
- **Real-World Simulation**: This mimics real VFL where parties don't know each other's data distributions

```
X1 = torch.tensor(scaler1.fit_transform(df[party1_features].values),
dtype=torch.float32)
X2 = torch.tensor(scaler2.fit_transform(df[party2_features].values),
dtype=torch.float32)
X3 = torch.tensor(scaler3.fit_transform(df[party3_features].values),
dtype=torch.float32)
```

- **Lines 68-70**:
  - Extracts features for each party from DataFrame
  - Fits scaler and transforms (normalizes) features
  - Converts to PyTorch tensors with float32 dtype

Why convert to PyTorch tensors?

- **GPU Acceleration**: Tensors can run on GPU for faster training
- **Automatic Differentiation**: PyTorch tracks gradients for backpropagation
- **Neural Network Compatibility**: PyTorch models require tensor inputs
- **Memory Efficiency**: float32 uses less memory than float64 while maintaining sufficient precision

```
y = torch.tensor(df[label_col].values, dtype=torch.float32)
```

- **Line 71**: Converts labels to PyTorch tensor

⚠ **IMPORTANT: Label Ownership in VFL**

In this simulation, labels are extracted separately, but **in real VFL, labels belong to the "Active Party"** (also called "Label Party" or "Coordinator Party").

In Real VFL:

- **Active Party** (typically Party 1, but can be any party):
  - Has BOTH their own features AND the labels
  - Runs the `ActiveClassifier`
  - Coordinates training by:
    1. Receiving embeddings from other parties
    2. Combining embeddings
    3. Computing loss using their labels
    4. Sending gradients back to other parties
- **Passive Parties** (other parties):
  - Have ONLY their features (no labels)
  - Run their `LocalEncoder` locally
  - Send embeddings to active party
  - Receive gradients for their encoder

In this code:

- Labels are kept separate (`y`, `y_train`, `y_test`) for simulation
- The `ActiveClassifier` class name indicates it runs on the active party
- In production, one party would have both `X1` (or X2/X3) AND `y` locally

Typical VFL Setup:

```
Party 1 (Active): Has X1 features + labels y → Runs ActiveClassifier
Party 2 (Passive): Has X2 features only → Runs LocalEncoder
Party 3 (Passive): Has X3 features only → Runs LocalEncoder
```

```
x_parts = [X1, X2, X3]
```

- **Line 73**: Combines all party features into a list for easier handling

---

Section 5: Train/Test Split

```
# ----------------------------
# 4. Train/Test Split
```

```
# ----------------------------
train_idx, test_idx = train_test_split(
    range(len(df)),
    test_size=0.2,
    random_state=42,
    stratify=df[label_col]
)
```

- **Lines 78-83**: Splits data indices:
  - 80% train, 20% test
  - `random_state=42`: Reproducibility
  - `stratify`: Maintains class distribution in both splits

Why train/test split?

- **Model Evaluation**: Test set provides unbiased estimate of model performance on unseen data
- **Overfitting Detection**: If train accuracy >> test accuracy, model is overfitting
- **Generalization**: Tests whether model learned general patterns, not just memorized training data
- **Standard Practice**: Industry standard for ML model validation

Why stratify?

- **Class Balance**: Ensures both train and test sets have similar attack/benign ratios
- **Prevents Bias**: Without stratification, one split might have all attacks, making evaluation meaningless
- **Realistic Evaluation**: Mimics real-world class distribution

```
x_train_parts = [x[train_idx] for x in x_parts]
x_test_parts = [x[test_idx] for x in x_parts]
y_train = y[train_idx]
y_test = y[test_idx]
```

- **Lines 85-88**: Applies train/test split to all party features and labels

Why split all parties together?

- **Consistent Indices**: Same samples must be in train/test for all parties (VFL requirement)
- **Alignment**: All parties must process the same flows to maintain correspondence
- **Real VFL**: In production, parties coordinate sample IDs to ensure alignment

---

Section 6: VFL Model Definition

```
# ----------------------------
# 5. VFL Model Definition
# ----------------------------
class LocalEncoder(nn.Module):
    def __init__(self, input_dim, embed_dim=32):
        super().__init__()
```

```
        self.net = nn.Sequential(
            nn.Linear(input_dim, embed_dim),
            nn.ReLU()
        )
```

- **Lines 93-100**: Defines `LocalEncoder` class:
  - Each party has its own encoder (runs locally)
  - `input_dim`: Number of features for this party
  - `embed_dim`: Size of embedding vector (default 32)
  - Architecture: Linear layer → ReLU activation
  - Output: Embedding vector of size `embed_dim`

Why Local Encoders?

- **Privacy**: Each party processes its own data locally - raw features never leave the party
- **Feature Extraction**: Encoders learn to extract meaningful patterns from party-specific features
- **Embedding Space**: Converts different feature spaces (per party) into a common embedding space
- **Non-linearity**: ReLU allows the model to learn complex, non-linear relationships
- **Distributed Computation**: In real VFL, each party runs its encoder on its own servers

```
    def forward(self, x):
        return self.net(x)
```

- **Lines 102-103**: Forward pass through the encoder network

```
class ActiveClassifier(nn.Module):
    def __init__(self, embed_dim=32):
        super().__init__()
        self.fc = nn.Linear(embed_dim, 1)
```

- **Lines 106-110**: Defines `ActiveClassifier`:
  - Runs on the "active party" (has labels)
  - Single linear layer: `embed_dim` → 1 (binary classification)

Why Active Classifier?

- **Label Ownership**: Only the active party has labels, so only they can compute loss and gradients
- **Privacy**: Passive parties (without labels) cannot infer labels from embeddings alone
- **Centralized Decision**: Combines all party embeddings into final prediction
- **Simplicity**: Single layer is sufficient after rich embeddings from encoders
- **VFL Architecture**: Standard VFL pattern - active party coordinates training

```
    def forward(self, h):
        return torch.sigmoid(self.fc(h))
```

- **Lines 112-113**: Forward pass:
  - Linear transformation → Sigmoid (outputs probability 0-1)

```
class VFLModel(nn.Module):
    def __init__(self, input_dims, embed_dim=32):
        super().__init__()
        self.embed_dim = embed_dim
        self.encoders = nn.ModuleList([LocalEncoder(dim, embed_dim) for dim in
input_dims])
        self.classifier = ActiveClassifier(embed_dim)
```

- **Lines 116-121**: Main VFL model:
  - `input_dims`: List of input dimensions for each party
  - Creates one `LocalEncoder` per party
  - Single `ActiveClassifier` for final prediction

```
    def forward(self, x_parts):
        embeddings = [enc(x) for x, enc in zip(x_parts, self.encoders)]  # list of
[B, D]
        h = torch.stack(embeddings).sum(dim=0)                           # [B, D]
        y_hat = self.classifier(h)                                       # [B, 1]
        return y_hat
```

- **Lines 123-127**: Forward pass:
  - **Line 124**: Each party encodes its features → list of embeddings `[B, embed_dim]` each
  - **Line 125**: Stacks embeddings and sums them (aggregation)
  - **Line 126**: Classifier produces final prediction
  - Returns: Predicted probabilities `[batch_size, 1]`

Why sum embeddings instead of concatenate?

- **Additive Combination**: Summing allows parties to contribute additively to the decision
- **Interpretability**: SHAP can attribute contributions per party when using sum aggregation
- **Dimensionality**: Sum keeps embedding dimension constant (32D), concatenation would be 96D
- **Efficiency**: Sum is faster and uses less memory than concatenation
- **VFL Standard**: Sum aggregation is common in VFL literature for party-level interpretability

Why sigmoid activation?

- **Binary Classification**: Sigmoid outputs probabilities between 0 and 1 (attack probability)
- **Differentiable**: Smooth gradient for backpropagation
- **Interpretable**: Output directly represents attack likelihood

```
    def get_party_embeddings(self, x_parts):
        self.eval()
        with torch.no_grad():
```

```
            embeddings = [enc(x) for x, enc in zip(x_parts, self.encoders)]
        return embeddings  # list: [B, D] for each party
```

- **Lines 128-133**: Helper method to extract embeddings:
  - Sets model to evaluation mode
  - Disables gradient computation (for inference)
  - Returns list of embeddings (one per party)

Section 7: Training Functions

```
# ----------------------------
# 6. Train & Evaluate Functions
# ----------------------------
def train_vfl(model, x_parts, y, epochs=100, lr=1e-3):
    criterion = nn.BCELoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)
```

- **Lines 138-140**: Training function setup:
  - `BCELoss`: Binary Cross-Entropy loss (for binary classification)
  - `Adam`: Optimizer with learning rate 0.001

```
    for epoch in range(epochs):
        model.train()
        y_hat = model(x_parts).squeeze()
        loss = criterion(y_hat, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

- **Lines 141-147**: Training loop:
  - **Line 142**: Set model to training mode
  - **Line 143**: Forward pass, squeeze removes extra dimension
  - **Line 144**: Compute loss
  - **Line 145**: Zero gradients
  - **Line 146**: Backpropagation
  - **Line 147**: Update weights

```
        if epoch % 10 == 0:
            print(f"[VFL] Epoch {epoch} | Loss: {loss.item():.4f}")
```

- **Lines 148-149**: Print loss every 10 epochs

```
def evaluate(model, x_parts, y):
    model.eval()
    with torch.no_grad():
        y_hat = model(x_parts).squeeze()
        y_pred = (y_hat > 0.5).int()
```

- **Lines 151-156**: Evaluation function:
  - **Line 152**: Set to evaluation mode
  - **Line 153**: Disable gradients
  - **Line 154**: Get predictions
  - **Line 155**: Convert probabilities to binary predictions (threshold 0.5)

```
    acc = accuracy_score(y.cpu(), y_pred.cpu())
    rec = recall_score(y.cpu(), y_pred.cpu())
    f1 = f1_score(y.cpu(), y_pred.cpu())
    print(f"[VFL] Accuracy: {acc:.4f} | Recall: {rec:.4f} | F1-score: {f1:.4f}")
    return acc, rec, f1
```

- **Lines 156-160**: Compute and print metrics:
  - `.cpu()`: Move tensors to CPU for sklearn
  - Returns accuracy, recall, F1-score

Section 8: Train VFL Model

```
# ----------------------------
# 7. Train VFL Model
# ----------------------------
embed_dim = 32
model = VFLModel(
    input_dims=[len(party1_features), len(party2_features), len(party3_features)],
    embed_dim=embed_dim
)
```

- **Lines 165-169**: Create VFL model:
  - Embedding dimension: 32
  - Input dimensions: [4, 4, 4] (4 features per party)

```
train_vfl(model, x_train_parts, y_train, epochs=100)
acc, rec, f1 = evaluate(model, x_test_parts, y_test)
```

- **Lines 171-172**: Train model and evaluate on test set

```
perf_df = pd.DataFrame({"Metric": ["Accuracy", "Recall", "F1"], "Value": [acc,
rec, f1]})
perf_df.to_csv("vfl_shap_performance.csv", index=False)
```

- **Lines 174-175**: Save performance metrics to CSV

Section 9: Build Party-Level Meta-Features

```
# ----------------------------
# 8. Build Party-Level Meta-Features
# ----------------------------
model.eval()
with torch.no_grad():
    train_embeds = model.get_party_embeddings(x_train_parts)  # list [N_train, D]
    test_embeds = model.get_party_embeddings(x_test_parts)    # list [N_test, D]
```

- **Lines 180-183**: Extract embeddings for train and test sets:
  - Each party's embeddings: `[N, embed_dim]` where N is number of samples

```
h1_train = train_embeds[0].mean(dim=1, keepdim=True)
h2_train = train_embeds[1].mean(dim=1, keepdim=True)
h3_train = train_embeds[2].mean(dim=1, keepdim=True)
```

- **Lines 185-187**: Compute mean embedding per party for training set:
  - Averages across embedding dimensions → single scalar per sample per party
  - `keepdim=True`: Keeps dimension for concatenation

```
h1_test = test_embeds[0].mean(dim=1, keepdim=True)
h2_test = test_embeds[1].mean(dim=1, keepdim=True)
h3_test = test_embeds[2].mean(dim=1, keepdim=True)
```

- **Lines 189-191**: Same for test set

```
X_train_meta = torch.cat([h1_train, h2_train, h3_train], dim=1)  # [N_train, 3]
X_test_meta  = torch.cat([h1_test,  h2_test,  h3_test],  dim=1)  # [N_test, 3]
```

- **Lines 193-194**: Concatenate party embeddings:
  - Creates meta-features: `[N, 3]` where each column is one party's aggregated embedding

---

Section 10: Meta-Model Training

```
# ----------------------------
# 9. Meta-Model on [h1, h2, h3]
# ----------------------------
class PartyMetaModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(3, 1)
    def forward(self, x_meta):
        return torch.sigmoid(self.fc(x_meta))
```

- **Lines 199-204**: Simple meta-model:
  - Takes 3 inputs (one per party)
  - Single linear layer → sigmoid
  - Purpose: Mimic VFL model using only party-level summaries

```
meta_model = PartyMetaModel()
criterion = nn.BCELoss()
optimizer = optim.Adam(meta_model.parameters(), lr=1e-3)
meta_epochs = 50
```

- **Lines 206-209**: Initialize meta-model, loss, optimizer, epochs

```
model.eval()
for epoch in range(meta_epochs):
    meta_model.train()
    optimizer.zero_grad()
    with torch.no_grad():
        y_hat_teacher = model(x_train_parts).squeeze()
    y_hat_meta = meta_model(X_train_meta).squeeze()
    loss = criterion(y_hat_meta, y_hat_teacher)
    loss.backward()
    optimizer.step()
```

- **Lines 211-220**: Knowledge distillation:
  - **Line 214**: VFL model predictions (teacher)
  - **Line 215**: Meta-model predictions (student)
  - **Line 216**: Loss = difference between teacher and student
  - Meta-model learns to approximate VFL using only party summaries

```
    if epoch % 10 == 0:
        print(f"[META] Epoch {epoch} | Distillation Loss: {loss.item():.4f}")
```

- **Lines 221-222**: Print loss every 10 epochs

```
# Meta-model fidelity
meta_model.eval()
with torch.no_grad():
    y_hat_teacher_test = model(x_test_parts).squeeze().cpu().numpy()
    y_hat_meta_test = meta_model(X_test_meta).squeeze().cpu().numpy()
meta_corr = np.corrcoef(y_hat_teacher_test, y_hat_meta_test)[0, 1]
print(f"[META] Correlation between VFL and meta-model outputs on test set:
{meta_corr:.4f}")
```

- **Lines 225-230**: Check how well meta-model matches VFL:
  - Computes correlation coefficient
  - Higher correlation = better approximation

Section 11: SHAP Explanation

```
# -----------------------------
# 10. SHAP on Party Meta-Features (KernelExplainer)
# -----------------------------
meta_model.eval()

bg_size = min(100, X_train_meta.shape[0])
bg_idx = torch.randperm(X_train_meta.shape[0])[:bg_size]
background = X_train_meta[bg_idx].detach().cpu().numpy()  # [bg_size, 3]
```

- **Lines 235-239**: Prepare background dataset for SHAP:
  - **Line 236**: Background size: min(100, training samples)
  - **Line 237**: Random sample of training indices
  - **Line 238**: Extract background samples (reference distribution)

```
def meta_predict(x_np):
    x_t = torch.tensor(x_np, dtype=torch.float32)
    with torch.no_grad():
        out = meta_model(x_t).detach().cpu().numpy().reshape(-1)
    return out
```

- **Lines 241-245**: Wrapper function for SHAP:
  - Converts numpy → tensor → prediction → numpy
  - SHAP requires numpy-compatible function

```
explainer = shap.KernelExplainer(meta_predict, background)
```

- **Line 247**: Create SHAP explainer:
  - `KernelExplainer`: Model-agnostic SHAP (works with any model)

  - Uses background samples as reference

```
n_explain = min(200, X_test_meta.shape[0])
X_explain = X_test_meta[:n_explain].detach().cpu().numpy()  # [n_explain, 3]
```

- **Lines 249-250**: Select samples to explain (first 200 test samples)

```
shap_values = explainer.shap_values(X_explain, nsamples=200)
shap_values = np.asarray(shap_values)  # [n_explain, 3]
```

- **Lines 252-253**: Compute SHAP values:
  - `nsamples=200`: Number of samples for Monte Carlo estimation
  - Result: `[n_explain, 3]` - SHAP value per party per sample

Section 12: Aggregate SHAP Values

```
# -----------------------------
# 11. Aggregate SHAP for tables
# -----------------------------
phi = shap_values  # [n_explain, 3]
phi_abs = np.abs(phi)
```

- **Lines 258-259**:
  - `phi`: SHAP values (can be positive or negative)
  - `phi_abs`: Absolute SHAP values (magnitude of contribution)

```
# Global mean |SHAP| and percentage per party
mean_phi_abs = phi_abs.mean(axis=0)              # [3]
mean_phi_pct = mean_phi_abs / mean_phi_abs.sum() # [3]
```

- **Lines 261-263**: Compute global statistics:
  - **Line 262**: Mean absolute SHAP per party (across all samples)
  - **Line 263**: Percentage contribution per party

```
print("\n=== Global SHAP Party Attributions (All Explained Flows) ===")
global_rows = []
for i, name in enumerate(party_names):
    m_abs = float(np.asarray(mean_phi_abs[i]))
    m_pct = float(np.asarray(mean_phi_pct[i]))
    print(f"{name}: mean |SHAP| = {m_abs:.6f}, mean contribution =
{m_pct*100:5.2f}%")
    global_rows.append({
```

```
        "Party": name,
        "Domain": party_domains[i],
        "Feature_Group": party_feature_groups[i],
        "Mean_abs_SHAP_All": m_abs,
        "Mean_contrib_All": m_pct,
    })
global_df = pd.DataFrame(global_rows)
global_df.to_csv("vfl_shap_global_summary.csv", index=False)
```

- **Lines 265-279**: Print and save global SHAP summary:
  - Shows which party contributes most overall
  - Saves to CSV

```
# Class-conditional SHAP (DDoS vs benign) on explained subset
y_test_np = y_test.cpu().numpy()
y_explain = y_test_np[:n_explain]

attack_mask = (y_explain == 1)
benign_mask = (y_explain == 0)
```

- **Lines 281-285**: Separate attack vs benign samples:
  - `attack_mask`: Indices where label = 1 (attack)
  - `benign_mask`: Indices where label = 0 (benign)

```
def compute_subset_shap(mask):
    if mask.sum() == 0:
        return np.full(3, np.nan), np.full(3, np.nan)
    phi_sub = phi_abs[mask]
    mean_phi_sub = phi_sub.mean(axis=0)
    mean_pct_sub = mean_phi_sub / mean_phi_sub.sum()
    return mean_phi_sub, mean_pct_sub
```

- **Lines 287-293**: Helper function:
  - Computes SHAP statistics for a subset (attack or benign)
  - Returns mean absolute SHAP and percentage per party

```
mean_phi_attack, mean_pct_attack = compute_subset_shap(attack_mask)
mean_phi_benign, mean_pct_benign = compute_subset_shap(benign_mask)
```

- **Lines 295-296**: Compute statistics separately for attacks and benign flows

```
print("\n=== SHAP Party Attributions (DDoS/attack flows) ===")
attack_rows = []
for i, name in enumerate(party_names):
```

```
    a_abs = float(np.asarray(mean_phi_attack[i]))
    a_pct = float(np.asarray(mean_pct_attack[i]))
    print(f"{name}: mean |SHAP| = {a_abs:.6f}, mean contribution =
{a_pct*100:5.2f}%")
    attack_rows.append({
        "Party": name,
        "Domain": party_domains[i],
        "Feature_Group": party_feature_groups[i],
        "Mean_abs_SHAP_Attack": a_abs,
        "Mean_contrib_Attack": a_pct,
    })
attack_df = pd.DataFrame(attack_rows)
attack_df.to_csv("vfl_shap_ddos_summary.csv", index=False)
```

- **Lines 298-312**: Print and save attack-specific SHAP summary

```
print("\n=== SHAP Party Attributions (Benign flows) ===")
benign_rows = []
for i, name in enumerate(party_names):
    b_abs = float(np.asarray(mean_phi_benign[i]))
    b_pct = float(np.asarray(mean_pct_benign[i]))
    print(f"{name}: mean |SHAP| = {b_abs:.6f}, mean contribution =
{b_pct*100:5.2f}%")
    benign_rows.append({
        "Party": name,
        "Domain": party_domains[i],
        "Feature_Group": party_feature_groups[i],
        "Mean_abs_SHAP_Benign": b_abs,
        "Mean_contrib_Benign": b_pct,
    })
benign_df = pd.DataFrame(benign_rows)
benign_df.to_csv("vfl_shap_benign_summary.csv", index=False)
```

- **Lines 314-328**: Print and save benign-specific SHAP summary

---

Section 13: Agent-Level Mitigation

```
# -----------------------------
# 11b. Agent-level mitigation summary
# -----------------------------
top_party_idx = int(np.nanargmax(mean_pct_attack))
top_party_name = party_names[top_party_idx]
top_party_domain = party_domains[top_party_idx]
top_party_action = party_actions[top_party_idx]
top_party_share = float(mean_pct_attack[top_party_idx]) * 100.0
```

- **Lines 331-336**: Find dominant party for attacks:

  - `nanargmax`: Index of maximum value (handles NaN)
  - Extracts party name, domain, suggested action, and contribution percentage

```
print("\n=== Agentic Mitigation Recommendation (Attack flows) ===")
print(f"Dominant party for DDoS decisions: {top_party_name} ({top_party_domain}) "
      f"with mean SHAP contribution ≈ {top_party_share:5.2f}%.")

print(f"Recommended primary mitigation: {top_party_action}.")
```

- **Lines 338-341**: Print recommendation

```
agent_rows = []
for i, name in enumerate(party_names):
    agent_rows.append({
        "Party": name,
        "Domain": party_domains[i],
        "Feature_Group": party_feature_groups[i],
        "Mean_contrib_Attack": float(mean_pct_attack[i]),
        "Suggested_Action": party_actions[i]
    })
agent_df = pd.DataFrame(agent_rows)
agent_df.to_csv("vfl_shap_agent_mitigation_summary.csv", index=False)
```

- **Lines 343-352**: Save agent mitigation summary to CSV

Section 14: Per-Flow Examples

```
# ----------------------------
# 12. Example per-flow SHAP with agent hints
# ----------------------------
print("\n=== Example per-flow party SHAP (first 5 explained flows) ===")
example_rows = []
for i in range(min(5, n_explain)):
    phi_i = np.asarray(phi[i]).reshape(-1)
    phi_abs_i = np.abs(phi_i)
    if phi_abs_i.sum() == 0:
        pct_i = np.zeros_like(phi_abs_i)
    else:
        pct_i = phi_abs_i / phi_abs_i.sum()
```

- **Lines 357-365**: Process first 5 flows:
  - Extract SHAP values for flow `i`
  - Compute absolute values and percentages

```
    label_i = int(y_explain[i])
    print(f"\nFlow {i} (label = {label_i}):")
    row = {
        "FlowIndex": i,
        "Label": label_i,
    }
```

- **Lines 367-371**: Get label and initialize row dictionary

```
    for j, name in enumerate(party_names):
        phi_j = float(phi_i[j])
        pct_j = float(pct_i[j])
        print(f"  {name} SHAP: {phi_j: .4f} | contribution: {pct_j*100:5.2f}%")
        row[f"{name}_SHAP"] = phi_j
        row[f"{name}_contrib"] = pct_j
```

- **Lines 373-378**: For each party, print SHAP value and contribution, store in row

```
    if label_i == 1:
        top_j = int(np.argmax(pct_i))
        row["Top_Party"] = party_names[top_j]
        row["Top_Domain"] = party_domains[top_j]
        row["Suggested_Action"] = party_actions[top_j]
        print(f"  -> Agent recommendation: {party_names[top_j]} "
              f"({party_domains[top_j]}) should {party_actions[top_j]}.")
    else:
        row["Top_Party"] = ""
        row["Top_Domain"] = ""
        row["Suggested_Action"] = ""
```

- **Lines 380-390**: If attack (label=1):
  - Find party with highest contribution
  - Add recommendation to row
  - Print recommendation
  - If benign, leave fields empty

```
    example_rows.append(row)

examples_df = pd.DataFrame(example_rows)
examples_df.to_csv("vfl_shap_flow_examples.csv", index=False)
```

- **Lines 392-395**: Save per-flow examples to CSV

CELL 1: Mitigation Policies

Section 15: Simulated Mitigation Policies

```
# ----------------------------
# 13. Simulated mitigation policies (for magazine table)
# ----------------------------
# We use:
#  - y_hat_teacher_test: VFL predicted probabilities on test set (already computed
above)
#  - y_test_np: true labels on test set
#  - phi, y_explain: SHAP values and labels for first n_explain test samples

print("\n=== Simulated Mitigation Policies ===")

# Ensure numpy arrays
y_scores = y_hat_teacher_test           # shape [N_test]
y_true   = y_test_np                    # shape [N_test]
```

- **Lines 1-12**: Setup:
  - Uses VFL predictions and true labels
  - Converts to numpy arrays

```
# --- Policy 1: Global threshold only ---
# Block any flow with predicted DDoS probability > 0.5
blocked_global = (y_scores > 0.5)
```

- **Lines 14-16**: **Policy 1**: Simple threshold:
  - Block if predicted probability > 0.5

```
# Attack = 1, Benign = 0
attack_mask_full = (y_true == 1)
benign_mask_full = (y_true == 0)
```

- **Lines 18-19**: Create masks for attacks and benign flows

```
attack_block_rate_global = (
    blocked_global[attack_mask_full].mean() if attack_mask_full.sum() > 0 else
np.nan
)
benign_block_rate_global = (
    blocked_global[benign_mask_full].mean() if benign_mask_full.sum() > 0 else
np.nan
)
```

- **Lines 21-26**: Compute block rates:

  - Attack block rate: % of attacks correctly blocked
  - Benign block rate: % of benign flows incorrectly blocked (false positives)

```
# --- Policy 2: Agentic SHAP-guided mitigation ---
# Use only first n_explain flows where we have SHAP (phi, y_explain)
# For each explained flow:
#   - If predicted as attack (y_scores > 0.5) AND true label = 1
#   - AND Party 3 has highest SHAP share and share > 0.7
#   -> blocked by enterprise agent.
#
# For benign flows, we check how many would be (incorrectly) blocked by same rule.

n_explain = min(n_explain, len(y_scores))
scores_explain = y_scores[:n_explain]
labels_explain = y_explain[:n_explain]   # already defined above for SHAP subsets
```

- **Lines 28-40**: **Policy 2** setup:
  - Uses only flows with SHAP values
  - Rule: Block if predicted attack AND Party 3 dominant AND share > 70%

```
# Compute per-flow SHAP contributions
phi_explain = phi[:n_explain]            # [n_explain, 3]
phi_abs_explain = np.abs(phi_explain)
phi_sum = phi_abs_explain.sum(axis=1, keepdims=True)
phi_sum[phi_sum == 0] = 1.0
pct_explain = phi_abs_explain / phi_sum  # SHAP share per party
```

- **Lines 42-47**: Compute per-flow SHAP percentages:
  - Normalize absolute SHAP values to percentages

```
# Conditions for agentic blocking
pred_attack = (scores_explain > 0.5)
true_attack = (labels_explain == 1)
true_benign = (labels_explain == 0)

top_party = np.argmax(pct_explain, axis=1)          # index of dominant party
top_share = pct_explain[np.arange(n_explain), top_party]
```

- **Lines 49-53**: Create conditions:
  - `pred_attack`: Predicted as attack
  - `true_attack`, `true_benign`: True labels
  - `top_party`: Index of party with highest SHAP per flow
  - `top_share`: SHAP percentage of dominant party

```
# Agentic rule: block if predicted attack AND Party 3 dominant AND share > 0.7
agent_block = (pred_attack & (top_party == 2) & (top_share > 0.7))
```

- **Line 56**: **Agentic blocking rule**:
  - Block if: predicted attack AND Party 3 (index 2) is dominant AND share > 0.7

```
# Compute rates over explained subset
attack_mask_explain = true_attack
benign_mask_explain = true_benign

attack_block_rate_agent = (
    agent_block[attack_mask_explain].mean() if attack_mask_explain.sum() > 0 else
np.nan
)
benign_block_rate_agent = (
    agent_block[benign_mask_explain].mean() if benign_mask_explain.sum() > 0 else
np.nan
)
```

- **Lines 58-65**: Compute agentic policy block rates

```
# Convert to percentages
attack_block_rate_global_pct = float(attack_block_rate_global * 100.0)
benign_block_rate_global_pct = float(benign_block_rate_global * 100.0)
attack_block_rate_agent_pct  = float(attack_block_rate_agent * 100.0)
benign_block_rate_agent_pct  = float(benign_block_rate_agent * 100.0)
```

- **Lines 67-70**: Convert rates to percentages

```
print("\nPolicy\t\t\tAttack blocked (%)\tBenign blocked (%)")
print(f"Global threshold
only\t{attack_block_rate_global_pct:6.2f}\t\t\t{benign_block_rate_global_pct:6.2f}
")
print(f"Agentic SHAP-
guided\t{attack_block_rate_agent_pct:6.2f}\t\t\t{benign_block_rate_agent_pct:6.2f}
")
```

- **Lines 72-74**: Print comparison table

```
# Save as CSV for magazine table
policy_rows = [
    {
        "Policy": "Global threshold only",
        "Attack_blocked_pct": attack_block_rate_global_pct,
```

```
        "Benign_blocked_pct": benign_block_rate_global_pct,
    },
    {
        "Policy": "Agentic SHAP-guided mitigation",
        "Attack_blocked_pct": attack_block_rate_agent_pct,
        "Benign_blocked_pct": benign_block_rate_agent_pct,
    }
]

policy_df = pd.DataFrame(policy_rows)
policy_df.to_csv("vfl_shap_mitigation_policies.csv", index=False)
```

- **Lines 76-90**: Save policy comparison to CSV

Summary

This notebook demonstrates:

1. **Vertical Federated Learning (VFL)**: Multiple parties collaborate without sharing raw data
2. **SHAP Explanations**: Explains which party contributes most to predictions
3. **Agentic Mitigation**: Uses SHAP to recommend which party should take action
4. **Policy Comparison**: Compares simple threshold vs. SHAP-guided blocking

The key insight: **Party 3 (Enterprise/edge gateway) dominates DDoS detection decisions**, suggesting mitigation should focus on firewall/ACL adjustments.